

```

from typing import Tuple
"""
field.py – The sphere projection field as a UMA observation operator.

The standing wave pressure field  $P_{nlm}(r, \theta, \phi)$  IS the projection medium.
Each point in the field maps to a spherical harmonic amplitude.
The pendulum sweeping through it samples these amplitudes sequentially.

In UMA terms:
    The sphere field IS the observation operator  $H$ .
     $H(z) = \sum_{lm} [ \langle Y_{lm} | z \rangle * P_{nlm}(r_{pend}(t)) ]$ 

    The pendulum position  $r_{pend}(t) = R * (\sin(\omega t), 0, \cos(\omega t))$ 
    sweeps a great circle through the standing wave.

    Each sample is an exact measurement of a spherical harmonic component.
"""

import math
import numpy as np
from scipy.special import spherical_jn, sph_harm_y as sph_harm
from typing import Tuple
from uma.sphere.geometry import SystemGeometry, AcousticSphereGeometry

class SphereProjectionField:
    """
    The acoustic sphere as a UMA observation/projection operator.

    Maps the UMA field state  $z$  to a spherical harmonic decomposition,
    sampled by the pendulum as it sweeps through the standing wave.
    """

    def __init__(self, geometry: SystemGeometry, n_harmonics: int = 4):
        self.geo = geometry
        self.n_harmonics = n_harmonics
        self._build_mode_table()

    def _build_mode_table(self):
        """Build table of (n,l,m) modes up to n_harmonics."""
        self.modes = []
        for n in range(self.n_harmonics):
            for l in range(n + 1):
                for m in range(-l, l + 1):

```

```

        self.modes.append((n, l, m))

def pressure(self, r: float, theta: float, phi: float, t: float) -> complex:
    """
     $P(r, \theta, \phi, t) = \sum_{nlm} [j_n(k_{nl}r) * Y_{lm}(\theta, \phi) * e^{i\omega t}]$ 

    The exact standing wave pressure at point  $(r, \theta, \phi)$  at time  $t$ .
    """
    P = 0.0 + 0j
    omega = 2 * math.pi * self.geo.mode.f
    k = self.geo.mode.k
    for n, l, m in self.modes:
        jn = spherical_jn(n, k * r)
        Ylm = sph_harm(m, l, phi, theta)
        P += jn * Ylm * np.exp(1j * omega * t)
    return P

def pendulum_sample(self, t: float) -> Tuple[float, complex]:
    """
    Sample the pressure field at the pendulum position at time  $t$ .

    Pendulum sweeps a great circle:
         $r = R$  (constant, pendulum length)
         $\theta = \pi/2$  (equatorial plane)
         $\phi = \omega_{\text{pend}} * t$ 

    Returns  $(\phi, P)$  – angle and pressure at that angle.
    """
    from typing import Tuple
    omega_pend = 2 * math.pi * self.geo.f_pendulum
    phi = omega_pend * t % (2 * math.pi)
    theta = math.pi / 2 # equatorial sweep
    r = self.geo.L_pendulum

    P = self.pressure(r, theta, phi, t)
    return phi, P

def observation_vector(self, t: float, n_samples: int = 64) -> np.ndarray:
    """
    Build observation vector by sampling pendulum arc over one period.

    This is the H operator that maps field state to measurements.
     $n_{\text{samples}}$  samples over one pendulum period.
    Returns real pressure amplitudes at each pendulum position.
    """
    T = self.geo.T_pendulum
    times = np.linspace(t, t + T, n_samples, endpoint=False)

```

```

        samples = []
        for ti in times:
            phi, P = self.pendulum_sample(ti)
            samples.append(P.real)
        return np.array(samples)

def planck_input(self, nu: float, T_star: float) -> float:
    """
     $B(\nu, T) = \frac{2h\nu^3}{c^2} * \frac{1}{(e^{(h\nu/kT)} - 1)}$ 

    Spectral radiance of input starlight.
    e is structural – already inside the signal.
    """
    H = 6.626e-34
    C = 3e8
    KB = 1.381e-23
    x = H * nu / (KB * T_star)
    return 2 * H * nu**3 / C**2 / (math.exp(x) - 1)

def interference_amplitude(self, t: float, nu: float) -> float:
    """
    Amplitude of starlight ⊕ fan-synthetic interference at time t.

    phi_star = hν/kT (from Planck, e is already here)
    phi_synthetic = ω_fan * t

    A = B(ν, T) * cos(phi_star - phi_synthetic)
    """
    H = 6.626e-34
    KB = 1.381e-23
    phi_star = H * nu / (KB * self.geo.T_star)
    phi_synthetic = 2 * math.pi * self.geo.f_fan * t
    B = self.planck_input(nu, self.geo.T_star)
    return B * math.cos(phi_star - phi_synthetic)

class SpherePendulum:
    """
    The pendulum as a geometric sampler of the sphere field.
    Period and length are exact – derived from Bessel zeros.
    """

    def __init__(self, geometry: SystemGeometry):
        self.geo = geometry
        self.t = 0.0
        self.history = []

```

```

def position(self, t: float) -> Tuple[float, float, float]:
    """
    (x, y, z) position of pendulum bob at time t.
    Sweeps a great circle in the equatorial plane.
    """
    from typing import Tuple
    omega = 2 * math.pi * self.geo.f_pendulum
    L = self.geo.L_pendulum
    x = L * math.cos(omega * t)
    y = L * math.sin(omega * t)
    z = 0.0
    return x, y, z

def sample_field(self, field: SphereProjectionField, t: float) -> dict:
    """Sample the sphere field at current pendulum position."""
    x, y, z = self.position(t)
    r = math.sqrt(x**2 + y**2 + z**2)
    theta = math.acos(z / (r + 1e-15))
    phi = math.atan2(y, x)

    P = field.pressure(r, theta, phi, t)
    return {
        "t": t, "x": x, "y": y, "z": z,
        "r": r, "theta": theta, "phi": phi,
        "P_real": P.real, "P_imag": P.imag, "P_abs": abs(P),
    }

from typing import Tuple

class SphereVenturi:
    """
    The Venturi with throat derived from Bessel zeros.
    r0 = R * j_n1 / pi - exact resonance with sphere modes.
    """

    def __init__(self, geometry: SystemGeometry):
        self.geo = geometry
        self.r0 = geometry.r0
        self.r_in = geometry.r_in
        self.G = geometry.G
        self._R = self._build_radial(32)

    def _build_radial(self, N: int) -> np.ndarray:
        x = np.linspace(0, 1, N, endpoint=False)
        X, Y = np.meshgrid(x, x)
        dx = np.abs(X - 0.5); dx = np.minimum(dx, 1 - dx)

```

```

dy = np.abs(Y - 0.5); dy = np.minimum(dy, 1 - dy)
return np.sqrt(dx**2 + dy**2)

def coupling_field(self) -> np.ndarray:
    """V(r, r0) - exact, derived from sphere geometry."""
    r0 = self.r0 / (self.geo.R) # normalize to [0,0.5]
    r_in = self.r_in / (self.geo.R)
    r_out = 0.42
    r_eff = np.maximum(self._R, r_in)
    throat = (r0 / r_eff)
    envelope = np.clip((r_out - self._R) / (r_out - r0 + 1e-8), 0, 1)
    return throat * envelope

def apply(self, psi: np.ndarray, psi_inject: np.ndarray,
          dt: float) -> np.ndarray:
    """Apply sphere-derived Venturi to field."""
    V = self.coupling_field()
    return psi + dt * self.G * V * psi_inject

```